

High-Precision Drone Hovering and Stabilization via Sparse Optical Flow on the Arduino UNO Q

The convergence of microprocessor-grade visual processing and real-time microcontroller performance on a single edge-compute platform has transformed the capabilities of micro aerial vehicles operating in GPS-denied environments¹. Traditional flight controllers struggle to balance the low-latency requirements of inner-loop attitude stabilization with the intensive computational demands of visual odometry and spatial mapping³. The Arduino UNO Q resolves this bottleneck through a hybrid, dual-architecture system topology that partitions safety-critical control and high-level vision pipelines¹.

To achieve precise hover control without relying on external positioning systems, a drone must continuously estimate its lateral drift⁴. This is accomplished by executing an optimized, tilt-compensated sparse optical flow model on the Arduino UNO Q³. By running a pyramidal Lucas-Kanade feature tracking algorithm on the board's embedded Linux application processor, translational velocities are calculated at high frame rates and passed directly to a real-time cascaded Proportional-Integral-Derivative (PID) controller executing on the dedicated hardware microcontroller³.

Physical and Logical Subsystem Architecture

The hardware footprint of the Arduino UNO Q maintains the classic UNO form factor while integrating two distinct computing domains⁵. High-level computing tasks are hosted on the Qualcomm Dragonwing QRB2210 System-on-Chip (MPU), which features a quad-core 64-bit

Arm Cortex-A53 CPU running at up to **2.0 GHz** alongside an Adreno 702 graphics processing unit². This subsystem operates a full Debian Linux distribution with upstream support, allowing standard Unix utilities, GStreamer capture pipelines, and Python-based computer vision frameworks to run natively on the vehicle⁵.

Deterministic, low-latency execution is managed by the STMicroelectronics STM32U585 microcontroller (MCU)⁵. Utilizing an Arm Cortex-M33 architecture operating at **160 MHz**, this real-time processor executes Compiled C++ Arduino sketches on top of the Zephyr Real-Time Operating System⁵.

The physical interfaces of the platform are segmented by voltage and processing domain, which requires careful electrical and logical mapping:

Interface Domain	Physical Subsystem	Logic Level	Operating System	Primary Target
------------------	--------------------	-------------	------------------	----------------

				Functionality
MPU (Qualcomm QRB2210)	Bottom JMEDIA Connector ⁹	1.8 V [cite: 9, 14]	Debian Linux ⁵	MIPI-CSI camera capture, optical flow, tilt correction ⁴
MPU (Qualcomm QRB2210)	Bottom JMISC Connector ⁹	1.8 V [cite: 9, 14]	Debian Linux ⁵	High-speed digital logic, parallel camera interface ⁹
MCU (STM32U585)	Traditional UNO Headers ⁵	3.3 V [cite: 9, 15]	Zephyr RTOS ¹²	IMU polling, cascaded PID loop, motor PWM signals ¹
MCU (STM32U585)	On-board Qwiic Port ⁷	3.3 V [cite: 12, 16]	Zephyr RTOS ¹²	Modulino Distance sensor polling (ToF VL53L4CD) ¹⁶

Inter-processor communication between the MPU and MCU is established over an internal high-speed UART interface¹⁹. Physically, the STM32U585 Low-Power UART (LPUART1) is routed directly to the QRB2210 Serial External 2 (SE2) UART port, mapped in the Linux filesystem as /dev/ttyHS1²⁰. This serial interface is driven at a default baud rate of

115200 bps ¹⁹.

At the software layer, the bidirectional communication link is managed by a background Linux daemon called arduino-router¹⁹. This router service orchestrates Remote Procedure Calls (RPC) using the MessagePack binary serialization format¹⁹. Developers access this interface via the Arduino_RouterBridge library on the MCU and the arduino.app_utils module on the MPU²⁴.

MIPI-CSI Camera Integration and Device Tree Configuration

To minimize latency and optimize CPU utilization on the QRB2210, the optical flow model must capture frames through a direct hardware path rather than a USB bus¹⁰. The bottom JMEDIA connector on the Arduino UNO Q exposes two physical MIPI-CSI-2 22-pin camera ports capable of driving standard Raspberry Pi camera modules, such as the Sony IMX219

(Raspberry Pi Camera V2)¹⁰. Accessing these hardware camera ports requires an expansion carrier, such as the official Arduino UNO Media Carrier or the Camboard UNO Q breakout board¹⁰.

Because the Qualcomm application processor operates in a highly customized embedded Linux environment, the MIPI-CSI camera interfaces and their associated Camera Control Interface (CCI) I2C channels are not enabled by default in the generic boot device tree¹⁰. The developer must edit the compiled device tree blob (DTB) loaded at boot to map the hardware interfaces correctly¹⁰.

The physical configuration is established by executing a compilation shell script, typically named `setup_device.sh`, on the Debian system¹⁰. This script modifies the device tree source to declare the active CSI lane configuration, camera clocking, and I2C address spaces for the IMX219 sensor, compiling the changes into a final file named `qrb2210-arduino-imola-camera-rpiv2-r1b.dtb`¹⁰.

To force the Debian system to load this custom device tree configuration during the boot sequence, the local configuration loader file must be modified¹⁰. The boot entries are located within the UEFI system partition¹⁰. The configuration is applied by opening a terminal connection via an Android Debug Bridge (ADB) shell or a secure remote shell (SSH) connection and editing the entry file¹¹:

```
Bash
```

At the end of this configuration file, the developer must append the direct path pointing to the compiled device tree blob¹⁰:

```
devicetree /dtb/r1b/qrb2210-arduino-imola-camera-rpiv2-r1b.dtb
```

Once saved, restarting the platform binds the camera hardware to the system¹⁰. The camera pipeline can be verified immediately using the high-performance qcam tool or by initiating a low-level test capture via GStreamer utilities¹⁰:

```
Bash
```

Mathematical Models for Optical Flow and Tilting Dynamics

Optical flow models assume that the physical brightness of a tracked pixel remains constant across consecutive image frames³². For a pixel located at coordinate (x, y) at time t with intensity $I(x, y, t)$, this relationship is expressed as³²:

$$I(x, y, t) = I(x + \delta x, y + \delta y, t + \delta t)$$

Approximating this identity with a first-order Taylor series expansion yields the standard optical flow gradient constraint³²:

$$I_x u + I_y v + I_t = 0$$

where $I_x = \frac{\partial I}{\partial x}$ and $I_y = \frac{\partial I}{\partial y}$ are the spatial image intensity gradients, $I_t = \frac{\partial I}{\partial t}$ is the temporal intensity gradient, and $u = \frac{dx}{dt}$ and $v = \frac{dy}{dt}$ are the horizontal and vertical components of pixel velocity³².

The pyramidal Lucas-Kanade algorithm tracks displacement across multiple resolution layers to accommodate larger spatial movements⁶. Under the assumption that a local window of pixels

W (typically sized 15×15) shares a uniform translational motion, the system of equations is solved using a least-squares formulation⁴:

$$\begin{bmatrix} u \\ v \end{bmatrix} = (A^T A)^{-1} A^T b$$

where A represents the accumulated spatial gradients of all n pixels within the local window W , and b represents the temporal gradients⁴:

$$A = \begin{bmatrix} I_x(p_1) & I_y(p_1) \\ I_x(p_2) & I_y(p_2) \\ \vdots & \vdots \\ I_x(p_n) & I_y(p_n) \end{bmatrix}, \quad b = \begin{bmatrix} -I_t(p_1) \\ -I_t(p_2) \\ \vdots \\ -I_t(p_n) \end{bmatrix}$$

This least-squares optimization outputs raw pixel displacements (u, v) relative to the image frame⁴.

However, a downward-facing camera mounted on a drone detects apparent movement whenever the vehicle tilts, even if it is hovering in a fixed spatial position³. To isolate true translational velocity, the system must mathematically decouple rotational motion from translational motion using high-rate gyroscopic feedback³.

The overall observed optical flow vector $\mathbf{u}_{\text{observed}} = [u, v]^T$ is modeled as the linear superposition of translation and rotation⁴:

$$\mathbf{u}_{\text{observed}} = \mathbf{u}_{\text{trans}} + \mathbf{u}_{\text{rot}}$$

Assuming a central perspective camera model operating near hover with a focal length of f pixels, the apparent flow vectors caused by the roll rate (ω_x), pitch rate (ω_y), and yaw rate (ω_z) are calculated using the following equations³:

$$u_{\text{rot}} = -f\omega_y + \frac{xy}{f}\omega_x - \frac{x^2}{f}\omega_y + y\omega_z$$

$$v_{\text{rot}} = f\omega_x - \frac{y^2}{f}\omega_x + \frac{xy}{f}\omega_y - x\omega_z$$

where (x, y) is the coordinate of the tracked feature relative to the image center. By assuming a narrow field of view and restricting flight parameters to near-hover conditions (where $x, y \ll f$), the higher-order geometric terms can be simplified³⁴. Under these conditions, the equations reduce to:

$$u_{\text{trans}} \approx u_{\text{observed}} + f\omega_y$$

$$v_{\text{trans}} \approx v_{\text{observed}} - f\omega_x$$

These isolated pixel velocities are scaled to physical metric velocities (v_x, v_y) in meters per

second using the drone's absolute altitude (Z) above the ground plane⁴:

$$v_x = \frac{u_{\text{trans}} \cdot Z}{f \cdot \Delta t} = \frac{(u_{\text{observed}} + f\omega_y) \cdot Z}{f \cdot \Delta t}$$

$$v_y = \frac{v_{\text{trans}} \cdot Z}{f \cdot \Delta t} = \frac{(v_{\text{observed}} - f\omega_x) \cdot Z}{f \cdot \Delta t}$$

where Δt represents the exact time elapsed between successive frames¹⁸. The distance Z is retrieved dynamically via the Modulino Distance module¹⁸. This module uses the STMicroelectronics VL53L4CD Time-of-Flight sensor, which measures absolute distances up to 1300 mm via a standard I2C connection¹⁶.

Cascaded Feedback Loops for Stable Hover and Altitude Hold

Stabilizing a multirotor platform using optical flow feedback requires a nested, cascaded control architecture³. Directly mapping raw position errors to motor signals often causes severe oscillations or complete loss of control³. This instability occurs because changes in position are achieved through accelerations, which depend entirely on the attitude angles of the drone³.





Inner Loop (Attitude and Rate Control)

The innermost control loop runs on the STM32U585 MCU at 200 Hz ¹. This loop tracks the target pitch and roll angles ($\theta_{\text{target}}, \phi_{\text{target}}$) and compares them with real-time IMU gyroscopic rate measurements to calculate motor output adjustments³.

Middle Loop (Velocity Control)

The velocity loop operates at 30 Hz , matching the frame rate of the optical flow model⁹. This loop processes the error between the desired velocities and the compensated velocities ($v_{x,\text{comp}}, v_{y,\text{comp}}$) received from the MPU¹. It outputs the target attitude angles required to generate the necessary lateral forces³:

$$\theta_{\text{target}} = K_{p,v} e_{v_x}(t) + K_{i,v} \int e_{v_x}(\tau) d\tau + K_{d,v} \frac{de_{v_x}(t)}{dt}$$

$$\phi_{\text{target}} = K_{p,v} e_{v_y}(t) + K_{i,v} \int e_{v_y}(\tau) d\tau + K_{d,v} \frac{de_{v_y}(t)}{dt}$$

Outer Loop (Position Control)

The outermost loop runs at 10 Hz to track specific coordinate coordinates. It integrates the metric velocities to calculate relative position drift, generating target velocity demands for the middle control loop³.

The gains for these nested loops must be tuned progressively, starting from the innermost attitude rate loop and moving outward to the position loop³⁴. The recommended gain values for a standard 250 mm wheelbase quadrotor platform are detailed in the following table:

Control ler Loop Layer	Target Freque ncy	Feedba ck Source	Output Parame ter	Proport ional Gain (Kp)	Integral Gain (Ki)	Derivati ve Gain (Kd)	Action Limit
Inner (Attitud e)	200 Hz [cite: 2]	Onboar d IMU ³	Motor PWM ³	1.20 s ⁻¹ [cite: 34]	0.05 s ⁻²	0.04 s	±100% [cite: 34]
Middle (Velocit y)	30 Hz [cite: 9]	Compe nsated Flow ¹	Target Attitude ³	0.15 V/;	0.05 V/;	0.02 V/;	±15.0° r [cite: 3]
Outer (Positio n)	10 Hz	Integrat ed Flow ³	Target Velocity ³	0.80 s ⁻¹	0.00 s ⁻²	0.10 s	±1.50 m

Runnable Hybrid Optical Flow and Flight Control Program

The complete system is organized into a modular structure within the Arduino App Lab environment³⁵. This setup runs Python-based vision tasks on the MPU and real-time control loops on the MCU¹².

Project Directory Structure

OpticalFlowStabilizer/

```
├── app.yaml
├── sketch.yaml
├── python/
│   └── main.py
├── sketch/
└── sketch.ino
```

Component Definition: sketch.yaml

YAML

```

profiles:
  default:
    platforms:
      - platform: arduino:zephyr
    libraries:
      - Arduino_RouterBridge (0.3.0)
      - Arduino_Modulino (1.1.0)
      - dependency: Arduino_RPClite (0.2.1)
      - dependency: ArxContainer (0.7.0)
      - dependency: ArxTypeTraits (0.3.2)
      - dependency: DebugLog (0.8.4)
      - dependency: MsgPack (0.4.2)
    default_profile: default

```

Application Descriptor: app.yaml

```

YAML
name: OpticalFlowStabilizer
version: 1.0.0
author: Embedded Aerospace Engineer
description: Real-time tilt-compensated optical flow and nested PID flight controller.

```

MPU Application Code: python/main.py

This script runs in the Debian environment on the Qualcomm QRB2210 processor⁵. It captures images from the camera, runs the Lucas-Kanade optical flow model, compensates for tilt using telemetry from the MCU, and sends the calculated velocities back to the controller⁴.

```

Python
import cv2
import numpy as np
import time

```

```
from arduino.app.utils import App_Bridge
```

```
FOCAL_LENGTH_PX = 320.0
```

```
IMAGE_WIDTH = 320
```

```
IMAGE_HEIGHT = 240
```

```
LK_PARAMS = dict
```

```
winSize= 15 15
```

```
maxLevel= 2
```

```
criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
```

```
|
```

```
FEATURE_PARAMS = dict
```

```
maxCorners= 30
```

```
qualityLevel= 0.06
```

```
minDistance= 12
```

```
blockSize= 7
```

```
|
```

```
class OpticalFlowEstimator:
```

```
def __init__(self):
```

```
    self.prev_gray = None
```

```
    self.prev_points = None
```

```
    self.last_time = time.time()
```

```
def process(self, frame, roll_rate, pitch_rate, altitude):
```

```
    curr_time = time.time()
```

```
    dt = curr_time - self.last_time
```

```
    self.last_time = curr_time
```

```
    if dt <= 0.0
```

```
        return 0.0, 0.0
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
    if self.prev_points is None or len(self.prev_points) < 6
```

```
        self.prev_points = cv2.goodFeaturesToTrack(gray, mask=None)
```

```
    FEATURE_PARAMS
```

```
    self.prev_gray = gray.copy()
```

```
    return 0.0, 0.0
```

```
    next_points, status, error = cv2.calcOpticalFlowPyrLK(
```

```
        self.prev_gray, gray, self.prev_points, None, **LK_PARAMS
```

```

    if next_points is None:
        self.prev_points = None
        return 0.0 0.0

    valid_prev = self.prev_points[status == 1]
    valid_next = next_points[status == 1]

    if len(valid_prev) == 0:
        self.prev_points = None
        return 0.0 0.0

    displacements = valid_next - valid_prev
    mean_displacement = np.mean(displacements, axis=0)
    u_observed = mean_displacement[0]
    v_observed = mean_displacement[1]

    u_compensated = u_observed + (FOCAL_LENGTH_PX * pitch_rate * dt)
    v_compensated = v_observed - (FOCAL_LENGTH_PX * roll_rate * dt)

    vx = (u_compensated * altitude) / (FOCAL_LENGTH_PX * dt)
    vy = (v_compensated * altitude) / (FOCAL_LENGTH_PX * dt)

    self.prev_gray = gray.copy()
    self.prev_points = valid_next.reshape(1 1 2)

    return float(vx) float(vy)

def loop():
    global cap, estimator

    ret, frame = cap.read()
    if not ret:
        return

    try:
        telemetry = Bridge.call("get_telemetry")
        if telemetry and len(telemetry) == 3:
            roll_rate, pitch_rate, altitude = telemetry
        else:
            roll_rate, pitch_rate, altitude = 0.0 0.0 0.1
    except Exception:

```

```

roll_rate, pitch_rate, altitude = 0.0 0.0 0.1

vx, vy = estimator.process(frame, roll_rate, pitch_rate, altitude)

if abs(vx) < 4.0 and abs(vy) < 4.0
    Bridge.notify("update_velocity" vx, vy)

if __name__ == "__main__":
    pipeline = (
        "libcamerasrc ! "
        "video/x-raw, width=320, height=240, framerate=30/1 ! "
        "videoconvert ! appsink"
    )
    cap = cv2.VideoCapture(pipeline, cv2.CAP_GSTREAMER)

    if not cap.isOpened():
        cap = cv2.VideoCapture(0)
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, IMAGE_WIDTH)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, IMAGE_HEIGHT)

    estimator = OpticalFlowEstimator()
    time.sleep(2.0)

    App.run(user_loop=loop)

```

MCU Real-Time Application Code: sketch/sketch.ino

This sketch executes on the STM32U585 MCU over the Zephyr RTOS core⁵. It polls the Modulino Distance sensor via the Qwiic I2C bus, manages the RPC bridge, and runs the high-frequency PID control loops¹.

```

C++
#include <Arduino_RouterBridge.h>
#include <Arduino_Modulino.h>

ModulinoDistance distanceSensor;

volatile float current_altitude_meters = 0.1f

```

```
volatile float gyro_roll_rate_rads = 0.0f  
volatile float gyro_pitch_rate_rads = 0.0f
```

```
volatile float estimated_vx = 0.0f  
volatile float estimated_vy = 0.0f  
volatile uint32_t last_velocity_update_ms = 0
```

```
const float KP_VELOCITY = 0.15f  
const float KI_VELOCITY = 0.05f  
const float KD_VELOCITY = 0.02f
```

```
const float KP_ATTITUDE = 1.20f  
const float KD_ATTITUDE = 0.05f
```

```
uint32_t last_loop_time_us = 0  
float vx_integral = 0.0f  
float vy_integral = 0.0f  
float prev_vx_error = 0.0f  
float prev_vy_error = 0.0f
```

```
void update_velocity_handler(float vx, float vy)  
{  
    estimated_vx = vx  
    estimated_vy = vy  
    last_velocity_update_ms = millis()  
}
```

```
msgpack::type::tuple<float, float, float> get_telemetry_handler()  
{  
    return msgpack::type::make_tuple(gyro_roll_rate_rads, gyro_pitch_rate_rads,  
    current_altitude_meters)  
}
```

```
void setup()  
{  
    Serial.begin(115200)
```

```
    if (!IBridge.begin())  
        while (true)  
            delay(1000)  
}
```

```
    Monitor.begin(115200)  
    Modulino.begin()
```

```

    if (!distanceSensor.begin())
        Monitor.println("Distance initialization failed.");
}

Bridge.provide_safe("get_telemetry" get_telemetry_handler);
Bridge.provide_safe("update_velocity" update_velocity_handler);

last_loop_time_us = micros();

}

void loop()
{
    uint32_t current_time_us = micros();
    float dt = (current_time_us - last_loop_time_us) / 1000000.0f;
    last_loop_time_us = current_time_us;

    if (dt <= 0.0f) return;

    // Gyros values should be read from an active IMU over SPI here
    gyro_roll_rate_rads = 0.01f;
    gyro_pitch_rate_rads = -0.01f;

    if (!distanceSensor.available())
        float raw_distance = distanceSensor.get();
        if (!isnan(raw_distance))
            current_altitude_meters = raw_distance / 1000.0f;
}

if (millis() - last_velocity_update_ms > 300)
{
    estimated_vx = 0.0f;
    estimated_vy = 0.0f;
}

float target_vx = 0.0f;
float target_vy = 0.0f;

float vx_error = target_vx - estimated_vx;
float vy_error = target_vy - estimated_vy;

vx_integral += vx_error * dt;
vy_integral += vy_error * dt;

vx_integral = constrain_value(vx_integral, -0.2f, 0.2f);

```

```

vy_integral = constrain_value(vy_integral, -0.2f, 0.2f)

float vx_derivative = (vx_error - prev_vx_error) / dt;
float vy_derivative = (vy_error - prev_vy_error) / dt;

prev_vx_error = vx_error;
prev_vy_error = vy_error;

float target_pitch_angle = (vx_error * KP_VELOCITY) + (vx_integral * KI_VELOCITY) +
vx_derivative * KD_VELOCITY;
float target_roll_angle = (vy_error * KP_VELOCITY) + (vy_integral * KI_VELOCITY) +
vy_derivative * KD_VELOCITY;

target_pitch_angle = constrain_value(target_pitch_angle, -0.26f, 0.26f);
target_roll_angle = constrain_value(target_roll_angle, -0.26f, 0.26f);

float pitch_rate_target = (target_pitch_angle - 0.0f) * KP_ATTITUDE;
float roll_rate_target = (target_roll_angle - 0.0f) * KP_ATTITUDE;

float motor_pitch_out = (pitch_rate_target - gyro_pitch_rate_rads) * KD_ATTITUDE;
float motor_roll_out = (roll_rate_target - gyro_roll_rate_rads) * KD_ATTITUDE;

delay(5);

float constrain_value(float val, float min_val, float max_val)
{
    if (val < min_val) return min_val;
    if (val > max_val) return max_val;
    return val;
}

```

Edge-Compute Bottlenecks and Mitigation Strategies

Operating complex software on a heterogeneous edge-compute platform introduces several critical timing and synchronization challenges¹:

Serialization and Transmission Latency

The arduino-router daemon uses MessagePack-RPC, which introduces serialization and deserialization overhead¹⁹. While this binary protocol is much faster than text-based formats like JSON, synchronous Bridge.call() executions block the calling thread²⁰. If the MPU script makes synchronous blocking calls during high-rate tracking, the optical flow pipeline can stall²⁶.

To minimize these latency spikes, the system uses synchronous Bridge.call() requests only to

retrieve low-volume telemetry data²⁰. The high-rate calculated metric velocities are streamed back to the MCU using non-blocking, asynchronous notifications via `Bridge.notify()`¹⁹. This preserves processing resources and ensures the real-time flight control loop remains responsive²⁵.

Shared Serial Port Buffering and Service Collisions

The physical serial port `/dev/ttyHS1` can experience buffer saturation or transmission collisions if multiple background processes attempt to access it simultaneously³⁷. A common issue occurs when the `arduino-router` daemon is running while another application, such as a processing framework or debug terminal, attempts to read or write directly to the same serial interface³⁷. This port contention results in "Device or resource busy" errors, blocking communication³⁷. To resolve these resource conflicts, any active `arduino-router` service must be stopped before attempting direct serial interactions or manual uploads³⁷. The service status can be managed via `systemd` command lines²³:

```
Bash
```

```
sudo systemctl stop arduino-router
```

After completing the serial configuration, the routing daemon must be restarted to restore the communication bridge between the processors²³:

```
Bash
```

```
sudo systemctl start arduino-router
```

Electrical Isolation and Logic Level Compatibility

An electrical hazard arises from the mismatched logic levels of the MPU and MCU subsystems⁹. The Qualcomm QRB2210 application processor pins and its bottom connector banks (JMEDIA and JMISC) operate on a 1.8 V logic domain⁹. Conversely, the STM32U585 MCU and the classic top headers operate on a 3.3 V logic domain⁹.

Connecting **3.3 V** or **5 V** signals directly to the MPU pins will cause permanent thermal damage to the Qualcomm processor⁹. Level translation hardware must be used whenever interfacing the MPU with external sensors or classic Arduino shields²⁸.



Recovery of System Bootloaders

If a custom application inadvertently overwrites the real-time program space, the board may lose its ability to load new user sketches³⁸. The platform's real-time environment runs a custom sketch loader as part of the Arduino Zephyr core³⁸. If this system loader is damaged or erased, it can be restored by flashing the bootloader via an ADB-connected PC³⁸:

```
Bash
adb shell arduino-cli burn-bootloader -b arduino-zephyr-unoq -P jlink
```

This recovery command uses the Qualcomm processor's built-in CMSIS-DAP debugger to directly re-flash the STM32U585 MCU over the internal SWD pins¹³. This diagnostic utility

enables recovery without requiring external hardware programmers¹³.

Conclusions

Integrating visual odometry and real-time flight control on the Arduino UNO Q provides a compact, robust solution for GPS-denied drone navigation¹. Running the computational tasks of camera frame acquisition, Lucas-Kanade optical flow, and tilt compensation on the Qualcomm QRB2210 processor⁴ prevents latency spikes on the real-time STM32U585 microcontroller¹.

To ensure stable flight, the system uses a cascaded PID architecture where the attitude loops are driven by velocity updates from the MPU¹. Combined with an absolute distance sensor for metric scaling¹⁸, this dual-processor approach provides highly precise, self-contained hovering and drift stabilization for modern micro aerial vehicles¹.

Works cited

1. Webinar: Arduino - From Board to Build: Using UNO Q and App Lab - DigiKey TechForum, <https://forum.digikey.com/t/webinar-arduino-from-board-to-build-using-uno-q-and-app-lab/65985>
2. (PDF) Arduino UNO Q with Qualcomm AI Chip: Enabling Next-Generation Edge Intelligence for Embedded AI Prototyping - ResearchGate, https://www.researchgate.net/publication/399652115_Arduino_UNO_Q_with_Qualcomm_AI_Chip_Enabling_Next-Generation_Edge_Intelligence_for_Embedded_AI_Prototyping
3. Advanced Drone Control Systems, Simulating Real-World Physics with Python - WiredWhite, <https://wiredwhite.com/drone-control-systems-python/>
4. Visual-Inertial Odometry system for a drone in C++ | by Rusa UB | Medium, <https://medium.com/@rustamubajduloev4/visual-inertial-odometry-system-for-a-drone-in-c-c84fee5f8564>
5. UNO Q - Arduino Documentation, <https://docs.arduino.cc/hardware/uno-q>
6. Pyramid implementation of the optical-flow tracking operation. LK... | Download Scientific Diagram - ResearchGate, https://www.researchgate.net/figure/Pyramid-implementation-of-the-optical-flow-tracking-operation-LK-optical-flow_fig3_276177105
7. Arduino UNO Q 2GB - Grobotronics, <https://grobotronics.com/arduino-uno-q.html?sl=en>
8. attitude and position control of a quadrotor using onboard vision system - Middle East Technical University, <https://etd.lib.metu.edu.tr/upload/12624740/index.pdf>
9. Arduino UNO Q with Qualcomm SoC and STM32 MCU for AI. - Electronics-Lab, <https://www.electronics-lab.com/arduino-uno-q-merges-mcu-control-with-qualcomm-soc-for-ai-edge-computing/>
10. RevLmt/camboard-uno-q: shield for uno q - GitHub, <https://github.com/RevLmt/camboard-uno-q>
11. Debian Linux Basics for UNO Q - Arduino Documentation,

- <https://docs.arduino.cc/tutorials/uno-q/debian-guide>
12. Arduino UNO Q 2GB Single Board Computer | Linux + Microcontroller AI SBC, <https://www.rajguruelectronics.com/ProductView?tokDatRef=MzM3Nzg=&tokenId=MTM=>
 13. laurenvil/Uno-QClaw - GitHub, <https://github.com/laurenvil/Uno-QClaw>
 14. Arduino® UNO Q - Farnell, <https://www.farnell.com/datasheets/4673827.pdf>
 15. Automating PCB Inspection | Arduino UNO Q | Part 1: Introduction | DesignSpark - RS Online, <https://www.rs-online.com/designspark/automating-pcb-inspection-with-arduino-uno-q-part-1-introduction>
 16. Arduino Modulino Modules - Getting Started - DroneBot Workshop, <https://dronebotworkshop.com/modulino/>
 17. MODULINO DISTANCE ARDUINO - Extension module | connection cable,module; Comp: VL53L4CDV0DH/1; ABX00102 | Transfer Multisort Elektronik - Tme.eu, <https://www.tme.eu/en/details/abx00102/arduino-solutions/arduino/modulino-distance/>
 18. Modulino Distance + UNO Q – Proximity Meter - Arduino Project Hub, <https://projecthub.arduino.cc/mario-r/modulino-distance-uno-q-proximity-meter-5e98cf>
 19. Arduino Uno Q Communication: A Deep Dive into RPC - myembeddedstuff.com, <https://myembeddedstuff.com/arduino-uno-q-rpc-explained>
 20. Bridge API Reference | Arduino Documentation, <https://docs.arduino.cc/software/app-lab/bridge/bridge-api/>
 21. UNO Q: Sharing Knowledge, Tips, and Practical Skills - #52 by GolamMostafa, <https://forum.arduino.cc/t/uno-q-sharing-knowledge-tips-and-practical-skills/1442313/52>
 22. ทำความรู้จักบอร์ด Arduino Uno Q: MPU + MCU - IoT Engineering Education, https://iot-kmutnb.github.io/blogs/arduino/arduino_uno-q/
 23. Working with Arduino Router RPC, <https://docs.arduino.cc/tutorials/uno-q/routerbridge-multilanguage/>
 24. Get Started with App development on Arduino UNO Q - Qualcomm, <https://www.qualcomm.com/developer/project/get-started-with-app-development-on-arduino-uno-q>
 25. Is there an cheat sheet for Bridge transfers? - UNO Q - Arduino Forum, <https://forum.arduino.cc/t/is-there-an-cheat-sheet-for-bridge-transfers/1429960>
 26. Router Bridge bug - UNO Q - Arduino Forum, <https://forum.arduino.cc/t/router-bridge-bug/1425617>
 27. Uno Q Headless Smart Camera - element14 Community, https://community.element14.com/products/roadtest/rv/roadtest_reviews/1909/uno_q_headless_smart_camera
 28. Arduino® UNO Media Carrier - Farnell, <https://www.farnell.com/datasheets/4775192.pdf>
 29. We just announced SEVEN new products, ready to expand your Arduino® UNO™ Q board,

<https://blog.arduino.cc/2026/03/27/we-just-announced-seven-new-products-ready-to-expand-your-arduino-uno-q-board/>

30. RevLmt/camboard-uno-q at learn.r1build.com - GitHub,
<https://github.com/RevLmt/camboard-uno-q?ref=learn.r1build.com>
31. Arduino UNO Q - Edge Impulse Documentation,
<https://docs.edgeimpulse.com/hardware/boards/arduino-uno-q>
32. Why Optical Flow Matters — From Classic Algorithms to RAFT | by Padmanabh Butala,
<https://medium.com/@padmanabhbutala03/optical-why-optical-flow-matters-from-classic-algorithms-to-raft-df735f095abe>
33. [CV2] Motion Detection and Tracking in OpenCV: Frame Delta, MOG2, and Optical Flow Explained - DEV Community,
<https://dev.to/jarvissan22/blog-cv2-video-and-motion-detection-and-tracking-j4c>
34. Attitude control of multirotor UAVs: cascade P/PID vs PI-like architecture,
<https://re.public.polimi.it/retrieve/e0c31c0f-1453-4599-e053-1705fe0aef77/BRESG01-19.pdf>
35. Arduino UNO Q: Getting Started (Features, Pinout, Examples),
<https://randomnerdtutorials.com/arduino-uno-q-getting-started/>
36. UNO Q - OpenPLC Forum,
<https://openplc.discussion.community/post/uno-q-13743731>
37. UNO-Q and PROCESSING 4 - App Lab - Arduino Forum,
<https://forum.arduino.cc/t/uno-q-and-processing-4/1433321>
38. Arduino UNO Q - Zephyr Documentation,
https://docs.zephyrproject.org/latest/boards/arduino/uno_q/doc/index.html